CO326 Edge AI + Industrial IoT Mini Project

Smart Lighting Control - Group 17

Final Report

Members:

E/20/089 Edirimanna Y.H.

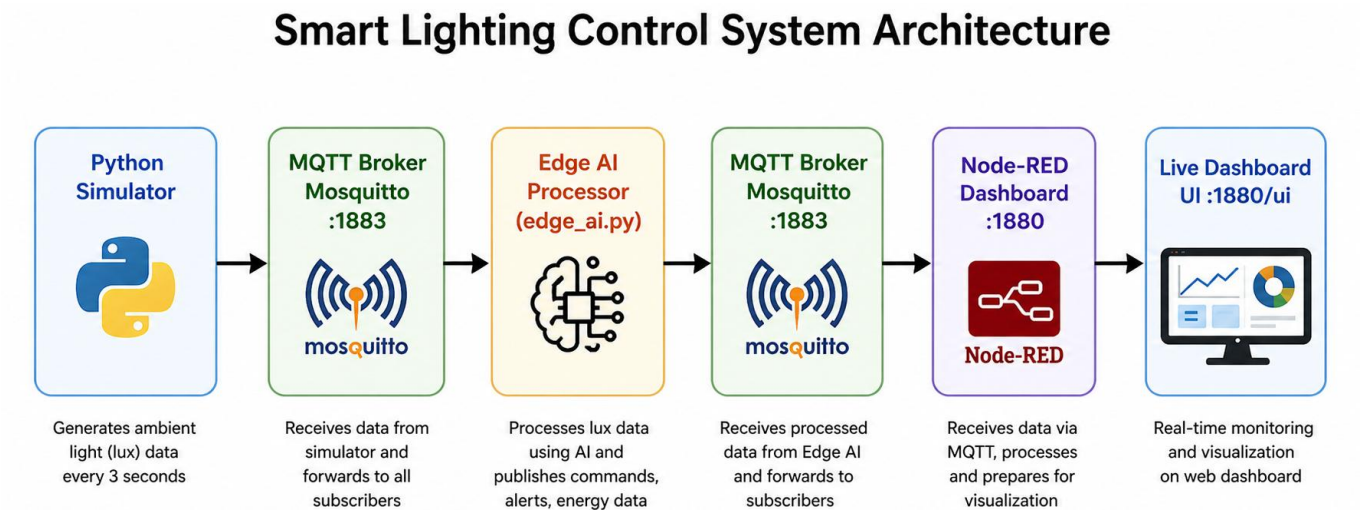
E/20/361 Senadheera Y.H.

E/20/366 Senavirathna A.P.B.P.

1.1 Project Overview

This project implements a Smart Lighting Control system using Edge AI and Industrial IoT technologies. The system simulates an ambient light sensor, processes the data locally using artificial intelligence, and automatically controls lighting brightness based on environmental conditions. The system is fully containerized using Docker and communicates via the MQTT protocol.

1.2 System Architecture Diagram



1.3 Docker Container Setup

The entire system runs inside 3 Docker containers managed by Docker Compose:

Container 1 - MQTT Broker (Mosquitto) Handles all message routing between the simulator and the AI processor. Runs on port 1883. Configured to allow anonymous connections for simplicity.

Container 2 - Node-RED Receives processed data from the AI and displays it on a live web dashboard. Runs on port 1880. Dashboard accessible at http://localhost:1880/ui

Container 3 - Python Edge Runs both the sensor simulator and the Edge AI processing scripts. Built from a custom Dockerfile using Python 3.11 with libraries: paho-mqtt, numpy, pandas, scikit-learn.

1.4 MQTT Topics

Topic	Direction	Purpose
sensors/group17/lighting/data	Simulator → AI	Raw lux sensor readings
sensors/group17/lighting/command	AI → Dashboard	Brightness control commands
sensors/group17/lighting/energy	AI → Dashboard	Energy saving calculations
alerts/group17/lighting/status	AI → Dashboard	Anomaly alert messages

1.5 Data Flow

Each message published to MQTT contains a JSON payload. For example, the sensor data topic publishes:

```
{ "lux": 738.45,
  "timestamp": 1777405144.92,
  "hour_sim": 14.3 }
```

The Edge AI subscribes to this topic, processes the data through all 5 AI features, and publishes results back to the appropriate topics every 3 seconds.

2.1 Why Simulation?

Since this project does not use physical hardware, a Python-based simulator was developed to generate realistic ambient light data. The simulator mimics the behaviour of a real LDR (Light Dependent Resistor) sensor by producing lux values that follow natural light patterns throughout the day.

2.2 Sinusoidal Day/Night Cycle

The core of the simulation uses a sine wave to model how natural light changes throughout a 24-hour period:

```
base_lux = 500 + 450 * math.sin((hour_of_day - 6) * math.pi / 12)
```

This produces:

- **Dawn (6am):** lux rising from near 0
- **Midday (12pm):** lux peaks around 950
- **Dusk (6pm):** lux falling back to 0
- **Night (12am):** lux near 0

2.3 Realistic Noise

Random Gaussian noise is added to every reading to simulate real sensor imperfections:

```
noise = random.uniform(-30, 30)
lux = base_lux + noise
```

This makes the data look like a real sensor rather than a perfect mathematical curve.

2.4 Anomaly Injection

Two types of anomalies are randomly injected to simulate real-world events:

Blackout Anomaly (4% probability per reading) Simulates sudden darkness such as a power cut, cloud cover, or room lights being switched off:

```
if random.random() < 0.04:
    lux = random.uniform(0, 50)
```

Flash Anomaly (2% probability per reading) Simulates sudden brightness such as a camera flash or security light activation:

```
if random.random() < 0.02:
    lux = random.uniform(900, 1100)
```

2.5 Data Rate

The simulator publishes one reading every 3 seconds. This rate was chosen to:

- Avoid overloading the MQTT broker
- Provide enough data for the AI rolling window (30 readings = 90 seconds of history)
- Keep the dashboard responsive and readable

2.6 Simulation Validation

The Lux Over Time chart on the Node-RED dashboard visually confirms the simulation is working correctly. The sinusoidal pattern is clearly visible with occasional sharp spikes representing the injected anomalies.

3.1 Overview

The Edge AI system implements 5 artificial intelligence features inside a single Python script (edge_ai.py). All processing happens locally on edge device before data is sent to the cloud dashboard, which core principle of Edge AI.

3.2 Feature 1 - Scene Detection

What it does: Classifies the current environment into one of four lighting scenes based on the simulated hour of day and lux value.

Scenes:

- Morning (6am–10am) - lux rising, moderate brightness needed
- Day (10am–5pm) - lux high, minimal artificial light needed
- Evening (5pm–10pm) - lux falling, increasing brightness needed
- Night (10pm–6am) - lux very low, full brightness needed

Why this technique: Rule-based classification is appropriate here because the relationship between time of day and lighting need is deterministic and well understood.

3.3 Feature 2 - Adaptive Learning

What it does: Instead of using fixed brightness thresholds, the system learns the average lux level for each scene over time and updates its decision thresholds automatically.

How it works:

- Maintains a rolling list of the last 20 lux readings per scene
- Calculates a moving average for each scene
- Uses the learned average as the brightness decision threshold
- Gets smarter the longer it runs

Why this technique: Moving average adaptive thresholding is a lightweight online learning technique that requires no training data and updates continuously, making it ideal for resource-constrained edge devices.

3.4 Feature 3 - Z-Score Anomaly Detection

What it does: Detects sudden unusual changes in lux that do not match the recent historical pattern, triggering alerts for potential real-world events.

How it works:

```
Z-score = (current_lux - mean of last 30 readings) / standard deviation
If Z-score > 3.0 → anomaly detected
If lux < mean → BLACKOUT alert
If lux > mean → FLASH alert
```

Why this technique: Z-score based anomaly detection is a well-established statistical machine learning technique for time-series data. It requires no labelled training data and adapts automatically to the current data distribution. A threshold of 3.0 standard deviations was chosen as it catches genuine anomalies while minimising false positives.

3.5 Feature 4 - Occupancy Prediction

What it does: Predicts whether the room is currently occupied or empty based on light patterns using a trained machine learning model.

Model: RandomForestClassifier from scikit-learn, trained on synthetic data with 3 input features:

- Current lux value
- Lux change from previous reading
- Simulated hour of day

Training data logic:

- Occupied rooms: moderate lux (150–500) with active changes, during working hours
- Empty rooms: very low lux (night) or very high stable lux (daylight through windows), minimal changes

Why Random Forest: Random Forest handles non-linear relationships between features, is robust to noise, requires minimal hyperparameter tuning, and works well with small datasets. It also provides interpretable feature importance scores.

3.6 Feature 5 - Energy Saving Calculator

What it does: Calculates in real time how much energy the smart lighting system is saving compared to always running lights at 100% brightness.

Calculation:

Baseline = 60W (standard bulb at 100%)
Actual Watts = 60W × (brightness% ÷ 100)
Saved Watts = Baseline - Actual Watts
Saving % = (Saved Watts ÷ Baseline) × 100
Total Saved (Wh) = accumulated over runtime

Why this matters: This feature demonstrates the tangible real-world value of the AI system and provides measurable evidence that Edge AI reduces energy consumption.

4.1 Dashboard Overview

The Node-RED dashboard runs at http://localhost:1880/ui and is divided into 3 sections providing a complete real-time view of the smart lighting system.

4.2 Light Sensor Section

Lux Over Time (Line Chart) Displays the last 10 minutes of lux readings as a continuous line chart. The sinusoidal pattern of the day/night cycle is clearly visible. Anomaly spikes appear as sharp vertical lines on the chart, visually confirming the anomaly injection is working correctly.

Ambient Light (Gauge) Displays the current lux reading on a gauge scaled from 0 to 1000 lux. Updates every 3 seconds with the latest sensor reading.

4.3 AI Control Section

Brightness Level (Gauge) Shows the current brightness command calculated by the AI as a percentage from 0 to 100%. During daytime with high ambient light, this reads low (20–40%) as the AI determines less artificial light is needed. During night scene it reads high (80–100%).

Scene (Text Display) Shows the currently detected scene: Morning, Day, Evening, or Night. This updates automatically as the simulated hour of day changes.

Occupancy (Text Display) Shows the Random Forest model's prediction: occupied or empty. The prediction is based on the current lux level, rate of change, and time of day.

4.4 Alerts & Energy Section

Energy Saved (Gauge) Displays the current energy saving percentage from 0 to 100%. When the system dims lights to 40% brightness this gauge reads 60%, representing 60% energy saved compared to running at full brightness.

Watts Saved (Text Display) Shows the exact wattage being saved in real time. For example if brightness is at 40%, this shows 36W saved out of the 60W baseline.

Alert Status (Text Display) Normally empty during regular operation. When the Z-score anomaly detector triggers, this displays either BLACKOUT - sudden darkness or FLASH - sudden brightness along with the lux value that triggered the alert.

4.5 Key Results Observed

- System correctly identifies Day scene during high lux periods and reduces brightness to 20–40%
- System correctly identifies Night scene and increases brightness to 80–100%
- Anomaly alerts trigger within one reading (3 seconds) of the anomaly being injected
- Energy savings of 40–80% are consistently observed during simulated daytime hours
- Occupancy prediction correctly identifies empty room during stable low-lux night periods

5.1 Challenges Faced

Challenge 1 - Docker Installation on Windows
Docker Desktop failed to install due to a Windows permissions error on C:\ProgramData\DockerDesktop. This was resolved by taking ownership of the folder using Command Prompt with administrator privileges and rerunning the installer.

Challenge 2 - Node-RED Gauge Display Issue
All dashboard gauges initially displayed raw JSON text instead of numeric values. This was because the gauge nodes received the full JSON payload object rather than the specific field value. The fix was to add function nodes between the JSON parser and each gauge/text widget to extract the correct field (e.g. msg.payload = msg.payload.lux).

Challenge 3 - MQTT Connection Between Containers
Initially the Python scripts used localhost as the broker address, which does not work inside Docker containers. The correct address is the Docker service name mqtt, which Docker's internal DNS resolves automatically between containers.

Challenge 4 - Anomaly Detection Tuning
The initial Z-score threshold of 2.0 produced too many false positive alerts due to the natural noise in the simulation. Increasing the threshold to 3.0 standard deviations reduced false positives significantly while still catching genuine anomalies.

Challenge 5 - Occupancy Model Training Data
Since no real occupancy data was available, synthetic training data was manually crafted based on logical assumptions about occupied vs empty room light patterns. While this works reasonably well for the simulation, real-world deployment would require data collected from actual sensors.

5.2 Future Improvements

Hardware Integration Replace the Python simulator with a real LDR sensor connected to an ESP32 or Raspberry Pi. The sensor would publish lux readings to the same MQTT topics, requiring no changes to the AI or dashboard code.

TensorFlow Lite Model Replace the Random Forest occupancy classifier with a TensorFlow Lite neural network trained on larger real-world datasets. TFLite models are optimized for edge devices and would provide better accuracy.

Mobile Push Notifications Integrate Telegram or WhatsApp API with Node-RED to send instant mobile notifications when anomalies are detected, making the alert system useful in a real building management context.

Historical Data Storage Add InfluxDB and Grafana to the Docker Compose stack for long-term data storage and historical analysis. This would allow energy savings to be tracked over days and weeks.

Multi-Room Scaling Extend the system to support multiple rooms by adding room identifiers to MQTT topics (e.g. sensors/group17/room1/data, sensors/group17/room2/data) and displaying each room separately on the dashboard.

Real Occupancy Feedback Add a PIR (Passive Infrared) motion sensor to provide ground truth occupancy labels, enabling the Random Forest model to be retrained on real data and improve prediction accuracy over time.